

Dokumentacja projektu

Fixo / Asystent Serwisanta

Dokumentacja użytkowa i techniczna aplikacji mobilnej oraz backendu

Element	Opis
Typ dokumentu	Dokumentacja łączona: użytkowa, techniczna oraz uruchomieniowa
Repozytorium	Monorepo: client/ - React Native/Expo, server/ - FastAPI
Wersja opracowania	1.0
Data	18 czerwca 2026

Celem dokumentu jest opisanie systemu z perspektywy użytkownika końcowego oraz osoby technicznej, która ma uruchomić, rozwijać lub testować projekt.

Spis treści

Zakres dokumentacji

- Cel projektu i zakres systemu
- Opis funkcjonalności aplikacji
- Instrukcja obsługi dla użytkownika
- Architektura systemu
- Frontend - React Native / Expo
- Backend - FastAPI
- API i komunikacja klient-serwer
- Baza danych i przetwarzanie dokumentów
- Konfiguracja, uruchomienie i testowanie
- Możliwe kierunki rozwoju

1. Cel projektu

Fixo jest aplikacją wspierającą pracę serwisanta wózków widłowych. System pozwala korzystać z dokumentacji technicznej w formie PDF, zadawać pytania w czacie oraz otrzymywać odpowiedzi generowane na podstawie fragmentów instrukcji powiązanych z wybranym urządzeniem.

Projekt składa się z aplikacji mobilnej oraz backendu. Aplikacja mobilna odpowiada za interfejs użytkownika, czat, wybór pojazdu, obsługę źródeł PDF i funkcje głosowe. Backend udostępnia REST API, panel administracyjny, mechanizmy RAG, transkrypcję mowy, syntezę odpowiedzi audio oraz integrację z bazą danych.

2. Główne funkcjonalności

- Wybór marki, typu oraz konkretnego modelu urządzenia serwisowego.
- Prowadzenie konwersacji w czacie powiązanej z wybranym urządzeniem.
- Wyszukiwanie informacji w zaimportowanych instrukcjach PDF.
- Wyświetlanie źródeł, z których korzysta odpowiedź asystenta.
- Obsługa transkrypcji mowy przez strumień audio oraz WebSocket.
- Synteza audio odpowiedzi asystenta i odtwarzanie jej po stronie aplikacji.
- Panel administracyjny do zarządzania markami, typami urządzeń, urządzeniami, dokumentami i rozmowami.

Instrukcja użytkowa

Jak korzystać z aplikacji

3. Role użytkowników

Rola	Zakres działań
Serwisant	Korzysta z aplikacji mobilnej, wybiera urządzenie, zadaje pytania i analizuje odpowiedzi asystenta.
Administrator	Zarządza danymi w panelu administracyjnym: markami, typami urządzeń, dokumentacją PDF, urządzeniami oraz rozmowami.

4. Scenariusz pracy serwisanta

- Użytkownik uruchamia aplikację mobilną i przechodzi do ekranu wyboru lub czatu.
- Wybiera markę, typ urządzenia oraz model, którego dotyczy problem serwisowy.
- W czacie opisuje problem tekstowo lub używa wejścia głosowego.
- Aplikacja wysyła pytanie do backendu wraz z kontekstem aktywnego wątku.
- Backend wyszukuje istotne fragmenty dokumentacji i generuje odpowiedź.
- Użytkownik czyta odpowiedź, może odtworzyć audio i sprawdzić powiązane źródła PDF.
- W razie potrzeby kontynuuje rozmowę w tym samym wątku, aby doprecyzować problem.

5. Ekran aplikacji mobilnej

Ekran / komponent	Przeznaczenie
HomeScreen / HomeActionPanel	Start pracy, wybór akcji oraz przejście do obsługi urządzenia.
ChatScreen	Główny widok rozmowy z asystentem serwisowym.
HistoryScreen	Przegląd historii rozmów i powrót do wcześniejszych wątków.
SettingsScreen	Ustawienia aplikacji i konfiguracja zachowania klienta.
VehicleFilters / VehicleCard	Filtrowanie i prezentacja dostępnych urządzeń.
SourcePanel / PdfViewer	Podgląd źródeł oraz instrukcji PDF użytych w odpowiedzi.
ControlPanel	Kontrolki rozmowy, mikrofonu, audio i wysyłania wiadomości.

Architektura systemu

Podział na klienta, backend i usługi zewnętrzne

6. Widok wysokiego poziomu

System działa w architekturze klient-serwer. Aplikacja mobilna komunikuje się z backendem przez HTTPS, REST API, Server-Sent Events oraz WebSocket. Backend korzysta z PostgreSQL z rozszerzeniem pgvector, aby przechowywać dane domenowe i embeddingi fragmentów dokumentacji.

Warstwa	Technologie	Odpowiedzialność
Klient	TypeScript, React Native, Expo, Expo Router	Interfejs użytkownika, czat, PDF, mikrofon i audio.
Backend	Python 3.12, FastAPI, SQLAlchemy, Alembic	API, autoryzacja Bearer, logika RAG, panel admina.
Baza danych	PostgreSQL + pgvector	Dane urzędzeń, wiadomości, dokumentów, fragmentów i wektorów.
AI / integracje	Azure OpenAI, OpenAI, Deepgram, Gemini TTS	Embeddingi, generowanie odpowiedzi, transkrypcja i synteza mowy.
Infrastruktura	Docker, Docker Compose, VPS, reverse proxy	Uruchomienie środowisk dev, staging i production.

7. Przepływ odpowiedzi RAG

- Aplikacja wysyła wiadomość użytkownika do backendu dla konkretnego wątku rozmowy.
- Backend tworzy embedding pytania z użyciem Azure OpenAI.
- System wyszukuje podobne fragmenty dokumentacji w PostgreSQL/pgvector i uzupełnia ranking wyszukiwaniem BM25.
- Model językowy otrzymuje prompt systemowy, historię rozmowy, znalezione fragmenty oraz pytanie użytkownika.
- Odpowiedź jest strumieniowana do klienta jako SSE, a po zakończeniu zapisywana w bazie.
- Powiązania między wiadomością i użytymi fragmentami są przechowywane w tabeli łączącej.

Backend

FastAPI, modele, endpointy i usługi

8. Struktura backendu

Ścieżka	Znaczenie
server/app/main.py	Konfiguracja aplikacji FastAPI, routerów, OpenAPI oraz middleware autoryzacji.
server/app/routers/	Endpointy REST, WebSocket i widoki panelu administracyjnego.
server/app/models/	Modele SQLAlchemy reprezentujące tabele bazy danych.
server/app/schemas/	Schematy Pydantic używane w API.
server/app/services/	Logika biznesowa: ingest, chunking, embedding, retrieval, LLM, STT, TTS.
server/alembic/	Migracje bazy danych.
server/tests/	Testy endpointów i usług.

9. Najważniejsze grupy endpointów

Prefiks	Zakres
/api/brands	Tworzenie, pobieranie, aktualizacja i usuwanie marek.
/api/device_types	Zarządzanie typami urządzeń.
/api/devices	Zarządzanie urządzeniami i ich powiązaniem.
/api/attachments	Dodawanie, pobieranie, łączenie i usuwanie dokumentów PDF.
/api/threads	Wątki, wiadomości oraz transmisja tekstu i audio.
/api/messages	Pobieranie informacji o źródłach użytych do utworzenia wiadomości.
/api/chunks	Dostęp do fragmentów dokumentacji oraz ich usuwanie.
/api/images	Pobieranie obrazów wyodrębnionych z dokumentacji.
/admin	Panel administracyjny HTML.

10. Autoryzacja

Backend zabezpiecza endpointy API nagłówkiem Authorization: Bearer <token>. Publiczne pozostają tylko /health, /docs, /redoc, /openapi.json oraz panel /admin.

11. Struktura aplikacji mobilnej

Ścieżka	Znaczenie
client/app/	Routing ekranów oparty o Expo Router.
client/components/	Komponenty UI: czat, panel źródeł, filtry pojazdów, PDF, kontrolki.
client/hooks/	Hooki logiki aplikacyjnej, m.in. API czatu, mikrofon, audio, ustawienia i dane pojazdów.
client/modules/	Natywne moduły Expo: audio stream oraz wake word.
client/utils/	Konfiguracja API, obsługa błędów i streaming czatu.
client/assets/	Ikony, grafiki, schematy i przykładowe dokumenty.
client/__tests__/	Testy jednostkowe i komponentowe.

12. Komunikacja z backendem

- REST API służy do pobierania danych słownikowych, urządzeń, historii i zasobów.
- SSE jest używane do strumieniowania odpowiedzi asystenta i zdarzeń audio.
- WebSocket obsługuje strumień transkrypcji mowy dla aktywnego wątku rozmowy.
- Konfiguracja adresu API znajduje się po stronie klienta w mechanizmach utils/api-config.

13. Funkcje głosowe

Projekt zawiera moduły obsługujące mikrofon, strumieniowanie audio, wykrywanie frazy aktywującej oraz odtwarzanie odpowiedzi. Po stronie backendu transkrypcja jest przekazywana do Deepgram, a odpowiedzi mogą być syntetyzowane jako audio przez usługę TTS.

Baza danych i dokumenty

Dane domenowe oraz przetwarzanie PDF

14. Model danych

Tabela / model	Opis
brands	Marki urządzeń, np. Toyota, Still, TCM.
device_types	Typy urządzeń, np. wózek widłowy lub inna kategoria sprzętu.
devices	Konkretne modele urządzeń wraz z marką, typem i opcjonalnym obrazem.
attachments	Załączone instrukcje PDF oraz nazwy oryginalnych plików.
chunks	Fragmenty tekstu wycięte z PDF wraz z embeddingiem i metadanymi.
chat_threads	Wątki rozmów przypisane do wybranego urządzenia.
messages	Wiadomości użytkownika i asystenta.
chunks_messages	Powiązania odpowiedzi z fragmentami dokumentów użytymi jako źródła.

15. Import i wyszukiwanie dokumentacji

- Administrator dodaje plik PDF jako attachment i łączy go z urządzeniem.
- Backend wyodrębnia treść dokumentu, dzieli ją na fragmenty i zapisuje metadane.
- Dla fragmentów generowane są embeddingi o rozmiarze zgodnym z modelem Azure OpenAI.
- Podczas rozmowy system wyszukuje fragmenty semantycznie oraz przez BM25.
- Odpowiedź asystenta zawiera treść wygenerowaną na podstawie znalezionych źródeł.

Uruchomienie i jakość

Konfiguracja, testy, deployment

16. Wymagania narzędziowe

Narzędzie	Wersja / opis
Node.js	24.14.0, zgodnie z .tool-versions
Python	3.12.13, zgodnie z .tool-versions
Poetry	2.3.2, zarządzanie zależnościami backendu
npm	Menedżer zależności klienta - projekt nie używa bun, pnpm ani yarn
Docker Compose	Środowisko bazy danych oraz deployment dev/staging/production

17. Uruchomienie backendu

```
cd server
make install
make dev
```

Dokumentacja OpenAPI jest dostępna pod adresem <http://localhost:8000/docs>. Przed uruchomieniem należy przygotować plik `server/.env` na podstawie `.env.example` i uzupełnić dane bazy oraz klucze usług zewnętrznych.

18. Uruchomienie klienta

```
cd client
make install
make android
```

Aplikacja korzysta z Expo i React Native. Wersja webowa może działać z ograniczoną funkcjonalnością, zwłaszcza w obszarze modułów natywnych, mikrofonu i odtwarzania audio.

19. Testy i formatowanie

Obszar	Komenda
Backend - lint	<code>cd server && make lint</code>
Backend - typecheck	<code>cd server && make typecheck</code>
Backend - testy	<code>cd server && make test</code>
Backend - format	<code>cd server && make format</code>
Frontend - testy	<code>cd client && make test</code>
Frontend - lint	<code>cd client && make lint</code>
Frontend - format	<code>cd client && make format</code>

Utrzymanie i rozwój

Dalsze prace

20. Deployment

Backend ma przygotowane osobne konfiguracje Docker Compose dla środowisk dev, staging i production. Wdrożenie produkcyjne działa pod domeną asystent-serwisanta.pl, a środowisko staging pod staging.asystent-serwisanta.pl. Środowiska są izolowane osobnymi kontenerami, wolumenami i portami.

21. Dobre praktyki utrzymaniowe

- Każda nowa funkcjonalność powinna mieć testy jednostkowe lub integracyjne, jeśli jest to uzasadnione.
- Migracji bazy danych wdrożonych na staging lub production nie należy wycofywać przez downgrade.
- Zmiany powinny przechodzić lint, typecheck, format-check oraz testy automatyczne.
- Dokumentacja powinna być aktualizowana razem ze zmianami API, modeli danych i zachowania aplikacji.
- Klucze API i dane połączeń powinny pozostać w plikach środowiskowych, poza kodem źródłowym.

22. Możliwe kierunki rozwoju

- Rozbudowa panelu administracyjnego o statystyki użycia i jakość odpowiedzi.
- Automatyczna dystrybucja aplikacji mobilnej jako APK lub przez store.
- Dokładniejsze cytowanie źródeł z numerami stron i podświetleniem fragmentów PDF.
- Obsługa większej liczby typów dokumentów serwisowych.
- Tryb offline dla wybranych instrukcji lub historii rozmów.
- Rozszerzenie testów manualnych i automatycznych dla funkcji audio.

23. Podsumowanie

Fixo łączy aplikację mobilną, backend FastAPI, bazę wektorową i usługi AI w narzędzie wspierające serwisanta podczas pracy z dokumentacją techniczną. Najważniejszą wartością systemu jest szybkie przejście od pytania użytkownika do odpowiedzi opartej na konkretnych instrukcjach przypisanych do danego urządzenia.